Document number:**P0542R5**
Date: **2018-06-08**
Audience: **Core Evolution Working Group**

Reply to: J. Daniel Garcia (josedaniel.garcia@uc3m.es)

Support for contract based programming in C++

G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup

1. Introduction

This paper is based on P0380 (P0380R1 [1]). All changes are relative to latest working draft (N4727 [2]).

2. Summary of wording

2.1. A new attributes syntax

To simplify the way that contracts are specified, and in line with our previous design paper, we propose a new syntax that may be used for attributes.

We propose this new syntax to be usable by attributes taking a single argument, which is a valid conditional expression.

```
[[contract-attribute: conditional-expression]]
```

where a *contract-attribute* is one of the following: **`expects`**, **`ensures`**, **`assert`**.

To avoid confusion, only the colon syntax for attributes is permitted in contract specifications

We also needed to support the idea of assertion levels needed by contracts. For this reason, we have introduced the concept of an attribute modifier that may appear after the attribute token itself.

```
[[contract-attribute modifier: conditional-expression]]
```

In this way we may specify contracts with assertion levels easily. We require that each attribute using the new proposed attribute syntax explicitly lists its accepted modifiers (if any).

In the case of contract attributes valid modifiers are: **`axiom`**, **`default`**, and **`audit`**

Finally, we needed to introduce the ability to define the return value to be used in postconditions. We allow, that an `ensures` attribute lists an identifier which is associated with the return value of a function.

```
[[ensures modifier identifier: conditional-expression]
```

With all this changes we can easily specify a contract:

```
int f(int x)
  [[expects audit: x>0]]
  [[ensures axiom res: res>1]];

void g() {
  int x = f(5);
  int y = f(12);
  //...
  [[assert: x+y>0]]
```

Note that while **`assert(expression)`** would expand as a function-like macro with the appropriate header, **`assert:`** is not a function-

like invocation, so does not expand.

2.2. Functions versus function types

The current standard establishes a distinction between an attribute applied to a function and to the function type (the normative text uses the form *"appertain to function type"*). With that approach there would be a difference between the following cases:

```
[[attribute]] void f(); // Attribute applied to function
void f [[attribute]] (); // Attribute applied to function
void f() [[attribute]]; // Attribute applied to function type
```

Only the third option is interesting for contract attributes as the preconditions and postconditions need make use of function's parameters.

```
void f(int x, int y)
  [[expects: x>0]]
  [[expects: y!=0]]
  [[ensures result: result > x+y]];
```

Consequently contracts attributes (as any other attribute in that syntactical location) *appertain to the function type*. However, they are not part of the function type.

Note that this does not solve the issue of being able to use attributes on lambda expressions (see Core issue 2097). In fact, until that issue is resolved it will not be possible to specify preconditions and postconditions for lambda expressions.

```
void f() {
  // Not currently supported
  auto increment = [](int x) [[expects: x>0]] { return x+1; };
  // ...
}
```

2.3 Contracts repetition

We require that any redeclaration of a function either has the contract or completely omits it.

```
int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]];

int f(int x); // OK. No contract.

int f(int x)
  [[expects: x>=0]]; // Error missing ensures and different expects condition

int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]]; //OK. Same contract.
```

Different argument names in redeclaration would be usually considered irrelevant.

```
int f(int x)
  [[expects: x>0]]
  [[ensures r: r>0]];

int f(int y)
  [[expects: y>0]]    // Should be OK
  [[ensures z: z>0]]; // Should be OK
```

Consequently, we require that contracts are the same in redeclarations. That means that:

- Preconditions and postconditions appear in the same order.
- Their modifiers are the same.
- Each conditional expression would satisfy the ODR if it appeared in function definition, except for renaming of parameters and return value identifiers.

However, we do not require this to be diagnosed.

2.4 Structured bindings and postconditions

One might imagine using structured bindings in postconditions:

```
std::tuple f()
  [[ensures [x,y]: x>0 && y.size()>0]];
```

However, we decided that this is something that might be considered for a future version. The same effect can be achieved currently as follows:

```
std::tuple f()
  [[ensures r: get<0>(r)>0 && get<1>(r).size()>0]];
```

2.5 Information of contract_violation

The information in **`contract_violation`** could be partially represented by a **`source_location`** object, from the Library Fundamentals V2 Technical Specification.

However, we defer this decision for a future version of this proposal.

2.6 Name lookup of contracts

Name lookup resolution in contracts may interact with the use of different build modes.

There are two answers here. The first one would be to make resolution dependent on the current build mode, requiring that only the contracts that would be evaluated in the current build mode need to parse correctly and pass the name lookup.

A second solution would be to make name lookup independent of build mode. In that case, all contracts would be required to pass name lookup independently of their assertion level. We have selected this second solution.

2.7 Identical contracts

We are also requiring in the wording that a redeclaration of a function that contains a contract needs to use the same contract (in the sense of ODR) that was present in the first declaration of the function.

The exact meaning would be to require contracts to be lexically the same token by token. This is a simpler definition, but would require that a redeclaration uses also the same names for functions parameters (if/when they are used in the contract).

A second solution would be to require the contracts to be logically equivalent which seems to introduce a number of additional implementation challenges.

A third option is to say that the functions must be textually equivalent except for a change of (parameter) variable names, but otherwise having the same structure. We have selected this option.

We have taken the route of requiring contract expressions to be odr-identical, except for the change of function parameter names. Diagnostic, however, is not required.

2.8 Throwing violation handler

If a violation handler throws an exception, it is necessary to clarify what is the effect when the continuation mode is *off*.

One option could be that the exception propagates as the handler did not *return*. However, that design option would open the opportunity for continuation when the continuation has been set to *off*.

Another design alternative would be to unconditionally invoke **`terminate()`**.

2.9 Additional information in contract violation

Besides original information in **`contract_violation`**, a new member has been added to store the **`assertion_level`**. This value contains a string with the *assertion-level* of the contract that has been violated.

2.10 Invoking the handler

The proposal does not support the direct invocation of the violation handler. Allowing so, would imply access to handler

supplied by the user.

Previous versions of this proposal included an **`always`** assertion level. That level was introduced as a way to make it possible to invoke the violation handler. However, the mechanism seemed to be problematic when used in interfaces. Addtionally, it resulted to be more controversial and not addressing exactly initial intent.

Consequently, we have removed the **`always`** level from the current proposal and we may revisit alternative solutions in the future.

2.11 Initialization of contract_violation objects

When a contract is broken, a **`contract_violation`** needs to be created with the corresponding information. There are several options on how such object should be populated with information.

Among the available options, one could be to leave this details as something to be defined by implementations. Otherwise, the exact population approach would need to be standardized.

This proposal establishes some constraints on the values that a contract violation object should hold.

- **Information on source location**:
  - In the case of a precondition violation the source location is implementation defined, although implementations are encouraged to report the caller site.
  - In the case of a postcondition violation the source location violation will be the one from the callee site.
  - In the case of an assertion, the source location will be the one from the assertion itself.
- **Comment**: It will contain an implementation-defined text relating to the *conditional-expression* of the contract that was not satisified.
- **Assertion level**: It will contain a string representation of the *assertion-level* of the contract that has failed.

2.12 Side effects in contracts

In general, contracts conditions are expected to be observable side-effect free. This implies that any observable effect in a contract condition leads to undefined behavior. This was the direction taken by EWG in Jacksonville.

Side effect is ill-formed	Side effect is undefined
8	16

This means that the modification of a global or a local in a contract expression is undefined behavior. This also includes reading a volatile object.

```
int x;
volatile int y;

void f(int n) [[expects: n>x]]; // OK
void g(int n) [[expects: n>x++]]; // Undefined behavior
void h(int n) [[expects: n++>0]]; // Undefined behavior
void j() {
  int n=3;
  [[assert: ++n>3]]; // Undefined behavior
  //...
}
void k() [[expects: y>0]]; // Undefined behavior
```

This also includes calling a function that potentially might modify a variable:

```
bool might_increment(int & x);

void f(int n) [[expects: might_increment(n)]]; // Undefined behavior

bool is_valid(int x) {
  std::cerr << "checking x\n";
  return x>0;
}

void g(int n) [[expects: is_valid(n)]]; // Undefined behavior
```

However, local side effects in functions invoked in the conditional expression are allowed. Please, note that they would not be observable from outside that function.

```
bool is_valid(int x) {
  int a=1;
  while (a<x) {
    if (x % a == 0) return true;
    a++;
  }
  return false;
}

void f(int n) [[expects: is_valid(x)]]
```

2.13 Location of a violation

The location of a violation belongs logically to the caller site. However, there is a number of cases where this might be difficult for implementers and users. Jason Merrill suggested to have an independent mechanism for obtaining the caller information (such as proposed stack trace in P0881). We propose that implementations should try to do their best to report the caller site and otherwise to report the callee site.

3. Questions about contracts programming

What is the effect of violating a contract while evaluating the check for another expression

Nothing special needs to be considered. When the first contract is violated the corresponding action is taken.

```
bool positive(const int * p) [[expects: p!=nullptr]] {
  return *p > 0;
}

bool g(int * p) [[expects: positive(p)]];

void test() {
  g(nullptr); // Contract violation when calling positive(nullptr)
}
```

Contract attribute as identifier

What happens if you try to use an identifier named as a contract attribute (e.g. **`requires`**, **`audit`**, ...)? Example:

```
X f(X & audit) [[ensures audit: audit.valid()]];
```

This is valid code. The first **`audit`** is interpreted as a *contract-level*. Then, the *conditional-expression* identifies the second **`audit`** correctly as the function argument.

4. Proposed Wording

In 5.10 **`[lex.name]`**, modify Table 4:

Table 4 — Identifiers with special meaning

`override`	**`final`**	**`axiom`**	**`audit`**

In 6.2/12 **`[basic.def.odr]`**, add a new bullet after bullet 12.5:

> - it is implementation-defined the conditions under which —if D contains an assertion, has a postcondition, or invokes a function with a precondition— each definition of D shall be translated using the same build level and violation continuation mode (10.6.11 **`[dcl.attr.contracts]`**); and

Modify clause 6.3.7 **`[basic.class.scope]`**, p. 1:

> The potential scope of a name declared in a class consists not only of the declarative region following the name's point of declaration, but also of all function bodies, default arguments, noexcept-specifiers, ~~and~~ default member initializers (12.2), and contract conditions (10.6.11 **`[dcl.attr.contracts]`**) in that class (including such things in nested classes).

Modify clause 6.4.1 **[basic.lookup.unqual]**, p. 7:

> A name used in the definition of a class X outside of a member function body, default argument, noexcept- specifier, default member initializer (12.2), contract condition (10.6.11 **[dcl.attr.contracts]**), or nested class definition shall be declared in one of the following ways:

Modify clause 6.4.1 **[basic.lookup.unqual]**, p. 8:

> For the members of a class X, a name used in a member function body, in a default argument, in a noexcept- specifier, in a default member initializer (12.2), in a contract condition (10.6.11 **[dcl.attr.contracts]**), or in the definition of a class member outside of the definition of X, following the member's declarator-id, shall be declared in one of the following ways:
> ...

Modify clause 8.6 **[expr.const]**, p. 2, adding a new bullet

- A checked contract (10.6.11 **[dcl.attr.contracts]**) whose predicate evaluates to false.

In 10.6.1/1 **[dcl.attr.grammar]**, modify the production for *attribute* as follows:

> ...
> *attribute-specifier:*
> **[[***attribute-using-prefix*$_{opt}$ *attribute-list***]]**
> *contract-attribute-specifier*
> *alignment-specifier*
> ...

Add a new section 10.6.11 Contracts attributes **[dcl.attr.contracts]**

1. Contract attributes are used to specify preconditions, postconditions, and assertions for functions.

contract-attribute-specifier:
[[`expects` *contract-level*$_{opt}$ *: conditional-expression* **]]**
[[`ensures` *contract-level*$_{opt}$ *identifier*$_{opt}$ *: conditional-expression* **]]**
[[`assert` *contract-level*$_{opt}$ *: conditional-expression* **]]**

contract-level:
`default`
`audit`
`axiom`

An ambiguity between a *contract-level* and an identifier is resolved in favor of *contract-level*.

2. A *contract-attribute-specifier* using `expects` is a *precondition*. It expresses a function's expectation on its arguments and/or the state of other objects using a predicate that is intended to hold upon entry into the function.

3. A *contract-attribute-specifier* using `ensures` is a *postcondition*. It expresses a condition that a function should ensure for the return value and/or the state of objects using a predicate that is intended to hold upon exit from the function.

4. A *contract-attribute-specifier* using `assert` is an *assertion*. It expresses a condition that is intended to be satisfied where it appears in a function body.

5. Preconditions, postconditions, and assertions are collectively called *contracts*. The *conditional-expression* in a contract is contextually converted to bool; the converted expression is called the *predicate* of the contract.

6. A *contract condition* is a precondition or a postcondition. A contract condition may be applied to the function type of a function declaration. The first declaration of a function shall specify all contract conditions (if any) of the function. Subsequent declarations shall either specify no contract conditions or the same list of contract conditions;

no diagnostic is required if corresponding conditions will always evaluate to the same value. The list of contract conditions of a function shall be the same if the declarations of that function appear in different translation units; no diagnostic required. If a friend declaration is the first declaration of the function in a translation unit and has a contract condition, the declaration shall be a definition and shall be the only declaration of the function in the translation unit.

7. Two lists of contract conditions are the same if they consist of the same contract conditions in the same order. Two contract conditions are the same if their contract levels are the same and their predicates are the same. Two predicates contained in *contract-attribute-specifier*s are the same if they would satisfy the one-definition rule (6.2 **[basic.def.odr]**) were they to appear in function definitions, except for renaming of parameters, return value identifiers (if any), and renaming of template parameters.

8. An assertion is checked by evaluating its predicate. The predicate of an assertion is potentially evaluated (6.2 **[basic.def.odr]**). The predicate of an assertion is evaluated as part of the evaluation of the null statement (9.2 **[stmt.expr]**) it applies to.

9. The predicate of a contract condition is potentially-evaluated (6.2 **[basic.def.odr]**) and has the same semantic restrictions as if it appeared as the first *expression-statement* in the body of the function it applies to. Additional access restrictions apply to names appearing in a contract condition of a member function of class C:

- Friendship is not considered (14.3 **[class.friend]**).
- For a contract condition of a public member function, no member of C or of an enclosing class of C is accessible unless it is a public member of C, or a member of a base class accessible as a public member of C (14.2 **[class.access.base]**).
- For a contract condition of a protected member function, no member of C or of an enclosing class of C is accessible unless it is a public or protected member of C, or a member of a base class accessible as a public or protected member of C.

For names appearing in a contract condition of a non-member function, friendship is not considered.

[*Example*:

```
class X {
public:
  int v() const;
  void f() [[expects: x>0]]; // Ill-formed
  void g() [[expects: v()>0]]; // OK
  friend void r(int z) [[expects: z>0]]; // OK
  friend void s(int z) [[expects: z>x]]; // Ill-formed
protected:
  int w();
  void h() [[expects: x>0]]; // Ill-formed
  void i() [[ensures: y>0]]; // OK
  void j() [[ensures: w()>0]]; // OK
  int y;
private:
  void k() [[expects: x>0]]; // OK
  int x;
};

class Y : public X {
public:
  void a() [[expects: v()>0]]; // OK
  void b() [[ensures: w()>0]]; // Ill-formed
protected:
  void c() [[expects: w()>0]]; // OK
};
```

— *end example*]

10. The only side effects of a predicate that are allowed in a *contract-attribute-specifier* are modifications of non-volatile objects whose lifetime began and ended within the evaluation of the predicate in functions invoked from that predicate. The behavior of any other side effect is undefined. [*Example*

```
void push(int x, queue & q)
  [[expects: !q.full()]]
  [[ensures: !q.empty()]]
{
  //...
  [[assert: q.is_valid()]];
```

```
  //...
}

int min=-42;
constexpr int max=42;

constexpr int g(int x)
  [[expects: min<=x]] // error
  [[expects: x<max]] // OK
{
  //...
  [[assert: 2*x < max]];
  [[assert: ++min > 0]]; // undefined
  //...
}
```

— *end example*]

11. If the *contract-level* of a *contract-attribute-specifier* is absent, it is assumed to be `default`. [*Note:* A `default` *contract-level* is expected to be used for those contracts where the cost of run-time checking is assumed to be small (or at least not expensive) compared to the cost of executing the function. An `audit` *contract-level* is expected to be used for those contracts where the cost of run-time checking is assumed to be large (or at least significant) compared to the cost of executing the function. An `axiom` *contract-level* is expected to be used for those contracts that are formal comments and are not evaluated at run-time. — *end note*].

12. A translation may be performed with one of the following *build levels*: *off*, *default*, or *audit*. A translation with build level set to *default* performs checking for `default` contracts. A translation with build level set to *audit* performs checking for `default` and `audit` contracts. If no build level is explicitly selected, the build level is *default*. The mechanism for selecting the build level is implementation-defined. The translation of a program consisting of translation units where the build level is not the same in all translation units is conditionally supported. There should be no programmatic way of setting, modifying, or querying the build level of a translation unit.

13. A precondition is checked by evaluating its predicate immediately before starting evaluation of the function body. [*Note*: the function body includes *function-try-blocks* (18 **[except]**) and *ctor-initializer* (15.6.2 **[class.base.init]**). — *end note*.] A postcondition is checked immediately before returning control to the caller of the function. [*Note*: The lifetime of local variables and temporaries has ended. Exiting via an exception and `longjmp` (21.112 **[csetjmp]**) are not considered returning control to the caller of the function. — *end note*]. An evaluation of a predicate that exits via an exception invokes **`std::terminate()`**.

14. If a function has multiple preconditions, their evaluation if any will be performed in the order they appear lexically. If a function has multiple postconditions, their evaluation if any will be performed in the order they appear lexically.

```
void f(int * p)
  [[expects: p!=nullptr]] // #1
  [[ensures: *p == 1]] // #3
  [[expects: *p == 0]] // #2
{
  *p = 1;
}
```

15. It is unspecified whether a contract that would not be checked under the current build level is evaluated. If it would evaluate to false the behavior is undefined. During constant expression evaluation (8.6 **[expr.const]**), contract evaluation is performed only for those contracts that are checked.

16. The violation handler of a program is a function of type "**noexcept**$_{\textbf{opt}}$ function of (lvalue reference to **const std::contract_violation**) returning **void**" specified in an implementation-defined manner. The violation handler is invoked when the predicate of a checked contract evaluates to false (called a contract violation). There should be no programmatic way of setting or modifying the violation handler. It is implementation defined how the violation handler is established for a program and how the **`std::contract_violation`** argument value is set. If a precondition is violated, the source location of the violation is implementation defined [*Note*: Implementations are encouraged but not required to report the caller site. — *end note*]. If a postcondition is violated, the source location of the violation is the source location of the function definition. If an assertion is violated, the source location of the violation is the source location of the statement to which the assertion is applied.

17. If a user-provided violation handler exits by throwing an exception and a contract is violated on a call to a function with a non-throwing exception specification, then the behavior is as if the exception escaped the function body. [*Note:* The function **`std::terminate()`** is invoked (18.5.1 **[except.terminate]**). — *end note*]

[*Example:*

```
void f(int x) noexcept [[expects: x>0]];

void g() {
  f(0); // std::terminate() if violation handler throws
  //...
}
```

— *end example*]

18. A translation may be performed with one of the following *violation continuation modes*: *off* or *on*. A translation with a violation continuation mode set to off terminates execution by invoking **`std::terminate()`** after completing the execution of the violation handler. A translation with a violation continuation mode set to *on* continues execution after completing the execution of the violation handler. If no continuation mode is explicitly selected, the default continuation mode is *off*. [*Note:* A continuation mode set to *on* provides the opportunity to install a logging handler to instrument a pre-existing code base and fix errors before enforcing checks. — *end note*] [*Example:*

```
void f(int x) [[expects: x > 0]];

void g() {
  f(0); // std::terminate() after handler if continuation mode is off
        // Proceeds after handler if continuation mode is on
  //...
}
```

— *end example*]

Add a new section 10.6.11.1 Interface contracts **`[dcl.attr.contracts.interface]`**

1. Multiple contract conditions may be applied to a function type with the same or different *contract-level*s. [*Example*:

```
int z;

bool is_prime(int k);

void f(int x)
  [[expects: x>0]]
  [[expects audit: is_prime(x)]]
  [[ensures: z>10]]
{
  //...
}
```

— *end example*]

2. A postcondition may introduce an identifier to represent the glvalue result or the prvalue result object of the function. [*Example:*

```
int f(char * c)
  [[ensures res: res>0 && c!=nullptr]];

int g(double * p)
  [[ensures audit res: res!=0 && p!=nullptr && *p<=0.0]];
```

— *end example*]

3. If a postcondition odr-uses a parameter value in its predicate and the function body makes direct or indirect modifications of that value the behavior is undefined. [*Example:*

```
int f(int x)
  [[ensures r: r==x]]
{
  return ++x; // Undefined behavior
}

int g(int * p)
```

```
  [[ensures r: p!=nullptr]]
{
  *p = 42; // OK. p is not modified
}

int h(int x)
  [[ensures r: r==x]]
{
  potentially_modify(x); // Undefined behavior if x is modified
  return x;
}
```

— *end example*]

4. [*Note*: A function pointer cannot include contract conditions. — *end note*]
[*Example*:

```
typedef int (*fpt)() [[ensures r: r!=0]]; // Ill-formed

int g(int x)
  [[expects: x>=0]]
  [[ensures r: r>x]]
{
  return x+1;
}

int (*pf)(int) = g; // OK
int x = pf(5); // contract conditions of g are checked
```

— *end example*]

5. If an overriding function specifies contract conditions, it shall specify the same list of contract conditions as its overridden functions; no diagnostic is required if corresponding conditions will always evaluate to the same value. Otherwise, it is considered to have the list of contract conditions from one of its overridden functions; the names in the contract conditions are bound, and the semantic constraints are checked, at the point where the contract conditions appear. Given a virtual function `f` with a contract condition that odr-uses `*this`, the class of which `f` is a direct member shall be be an unambiguous and accessible base class of any class in which `f` is overridden. If a function overrides more than one function, all of the overridden functions shall have the same list of contract conditions (10.6.11 [**`dcl.attr.contracts`**]); no diagnostic is required if corresponding conditions will always evaluate to the same value.

[*Example*:

```
struct A {
  virtual void g() [[expects: x==0]];
  int x = 42;
};

int x = 42;
struct B {
  virtual void g() [[expects: x==0]];
}

struct C : A, B {
  virtual void g(); // Ill-formed, no diagnostic required
};
```

— *end example*]

Modify clause 12.2 [**`class.mem`**], p. 6:

> A class is considered a completely-defined object type (6.7) (or complete type) at the closing } of the class-specifier. Within the class member-specification, the class is regarded as complete within function bodies, default arguments, noexcept-specifiers, ~~and~~ default member initializers, and contract conditions (10.6.11 [**`dcl.attr.contracts`**]) (including such things in nested classes). Otherwise it is regarded as incomplete within its own class member-specification.

Add at the end of clause 13.3 [**`class.virtual`**]:

18. [*Note*: For contracts, see 10.6.11.1 [**`dcl.attr.contracts.interface`**]. — *end note*]

Modify clause 17.8.2 **[temp.explicit]**, p. 5

> The declaration in an explicit-instantiation and the declaration produced by the corresponding substitution into the templated function, variable, or class are two declarations of the same entity. An *explicit-instantiation* of a function template shall not specify a contract condition (10.6.11 **[dcl.attr.contracts]**). [Note: These declarations are required to have matching types as specified in 6.5, except as specified in 18.4. [Example:

Add a note to 17.8.3 **[temp.expl.spec]**

> [*Note*: For an explicit specialization of a function template, the contract conditions (10.6.11 **[dcl.attr.contracts]**) of the explicit specialization are independent of those of the primary template. — *end note*]

Change 18.5.1 **[except.terminate]** adding a bullet after 1.6

> - when evaluation of a predicate (10.6.11 **[dcl.attr.contracts]**) exits via an exception.
> - when the violation handler invoked for a failed contract condition (10.6.11 **[dcl.attr.contracts]**) check on a noexcept function exits via an exception.

Modify clause 20.5.1.2/2 **[headers]**, Table 16

Table 16 — C++ library headers

…
<codecvt>
<contract>
<compare>
…

Modify clause 21.1/2 **[support.general]**:

> 2. The following subclauses describe common type definitions used throughout the library, characteristics of the predefined types, functions supporting start and termination of a C++ program, support for dynamic memory management, support for dynamic type identification, support for contract violation handling, support for exception processing, support for initializer lists, and other runtime support, as summarized in Table 32.

Table 32 — Language support library summary

	Subclause	Header(s)
21.2	Common definitions	<cstddef> <cstdlib>
21.3	Implementation properties	<limits> <climits> <cfloat>
21.4	Integer types	<cstdint>
21.5	Start and termination	<cstdlib>
21.6	Dynamic memory management	<new>
21.7	Type identification	<typeinfo>
21.8	Contract violation handling	<contract>
21.~~8~~9	Exception handling	<exception>
21.9~~10~~	Initializer lists	<initializer_list>
21.~~10~~11	Other runtime support	<csignal> <csetjmp> <cstdalign>

			<cstdarg> <cstdbool> <cstdlib>

Add a new section 21.9 **[support.contract]**, after 21.8 **[support.exception]**

21.9 Contract violation handling

1. The header `<contract>` defines a type for reporting information about contract violations generated by the implementation.

21.9.1 Header <contract> synopsys

```
namespace std {
  class contract_violation;
}
```

21.9.2 Class `contract_violation`

```
namespace std {
  class contract_violation {
  public:
    uint_least32_t line_number() const noexcept;
    string_view file_name() const noexcept;
    string_view function_name() const noexcept;
    string_view comment() const noexcept;
    string_view assertion_level() const noexcept;
  };
}
```

1. The class `contract_violation` describes information about a contract violation generated by the implementation.

```
uint_least32_t line_number() const noexcept;
```

2. *Returns:* The source code location where the contract violation happened (10.6.11/15 **[dcl.attr.contracts]**). If the location is unknown, an implementation may return `0`.

```
string_view file_name() const noexcept;
```

3. *Returns:* The source file name where the contract violation happened (10.6.11/15 **[dcl.attr.contracts]**). If the file name is unknown, an implementation may return `string_view{}`.

```
string_view function_name() const noexcept;
```

4. *Returns:* The name of the function where the contract violation happened (10.6.11/15 **[dcl.attr.contracts]**). If the function name is unknown, an implementation may return `string_view{}`.

```
string_view comment() const noexcept;
```

5. *Returns:* Implementation-defined text describing the predicate of the violated contract.

```
string_view assertion_level() const noexcept;
```

6. *Returns:* Text describing the *assertion-level* of the violated contract.

Acknowledgements

Thanks to Jens Maurer for his gigantic help in the detailed wording.

Jason Merrill suggested to express preconditions source location in terms of best effort. Hubert Tong helped with better wording for interaction with ODR. Daveed Vandevoorde made remarks on evaluation of conditional expressions and side effects.

Roger Orr, Andrzej Krzemieński, and Walter Brown reviewed the latest version of this document.

Alexander Beels, Andrzej Krzemienski and Melissa Mears reviewed previous versions of this document.

References

[1] G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup. A contract design. P0380R1. July, 2016.
[2] Richard Smith. Working Draft, Standard for Programming Language C++. N4727 October, 2017.